PHASE 2 OF 18

Language Basics

Variables, data types, literals, type casting and conversion, comments, identifiers, keywords, naming conventions and operators

IN THIS PHASE

2.1 Variables
2.2 Data Types
2.3 Type Conversion and Casting
2.4 Reading Console Input
2.5 Comments
2.6 Identifiers and Keywords
2.7 Naming Conventions
2.8 Operators
2.9 Operator Precedence
2.10 Predict the Output
2.11 Interview Questions
2.12 Quick Revision

2.1 Variables

A **variable** is a named memory location that stores a value. Java is **statically typed**: every variable has a type, fixed at compile time.

Storing data in a variable takes two steps, which can be combined:

```
byte age;          // 1. Declaration: type + name
age = 20;          // 2. Initialization: store a value
byte age2 = 20;    // both in one statement

int x = 10, y = 20, z;    // several variables of the same type in one line
```

Three kinds of variables (detailed in Phase 5)

Kind	Declared	Lives in	Default value?
Local	Inside a method, constructor or block	Stack frame	**No**. Must be initialised before use.
Instance	Inside a class, outside methods (no `static`)	Inside each object (Heap)	Yes (0, false, null, ...)
Static (class)	Inside a class with `static`	One copy per class	Yes

```
public class Demo {
    int count;                // instance: default 0
    static String bank;       // static: default null

    void show() {
        int local;
        // System.out.println(local);  // COMPILE ERROR: variable might not be initialized
        local = 5;
        System.out.println(local + count);
    }
}
```

Constants with final

```
final int MIN_AGE = 21;              // cannot be reassigned
// MIN_AGE = 25;                     // COMPILE ERROR
static final double PI = 3.14159;    // class-level constant (typical style)
```

CORRECTED FROM ORIGINAL NOTES

The original notes show `int MIN_AGE = 21;` as a constant. Naming it in capitals does **not** make it constant; only `final` does.

Local variable type inference: var (Java 10+)

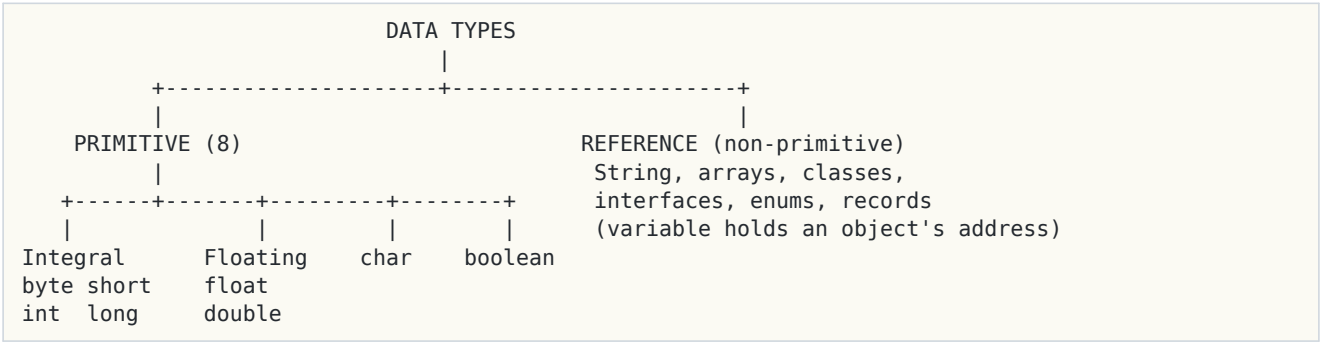
```
var count = 10;                 // inferred as int
var name = "Rohit";             // inferred as String
var list = new ArrayList<String>();   // inferred as ArrayList<String>
for (var entry : map.entrySet()) { }  // handy for long generic types
```

REMEMBER

Rules for `var`:

- Only for **local** variables (including loop variables), never fields, parameters or return types.
- Must be initialised on the same line: `var x;` and `var x = null;` do not compile.
- Type is still static: `var x = 10; x = "hi";` is a compile error.
- `var` is a reserved type name, not a keyword, so a variable can still be called `var`.

2.2 Data Types



The eight primitive types

Type	Size	Range	Default	Literal example
`byte`	8 bits (1 byte)	-128 to 127	0	`byte b = 100;`
`short`	16 bits (2 bytes)	-32,768 to 32,767	0	`short s = 30000;`
`int`	32 bits (4 bytes)	about -2.1 billion to 2.1 billion (-2^31 to 2^31-1)	0	`int i = 1_000_000;`
`long`	64 bits (8 bytes)	-2^63 to 2^63-1 (about 9.2 x 10^18)	0L	`long l = 30000000000L;`
`float`	32 bits	about ±3.4 x 10^38, **6–7 significant digits**	0.0f	`float f = 25.01f;`
`double`	64 bits	about ±1.8 x 10^308, **15–16 significant digits**	0.0	`double d = 25.01;`
`char`	16 bits (unsigned)	`'\u0000'` to `'\uffff'` (0 to 65,535)	`'\u0000'`	`char c = 'A';`
`boolean`	not specified by the JVM spec	`true` / `false`	false	`boolean ok = true;`

CORRECTED FROM ORIGINAL NOTES

Fixes to the original notes:

- `float` / `double` precision is about 6–7 / 15–16 **significant digits** in total, not "decimal places".
- `boolean` does not occupy "1 bit". The JVM specification leaves its size unspecified; in practice a `boolean` field usually takes 1 byte, and `boolean` locals are handled as `int`s.
- A `char` stores a **UTF-16 code unit** (Unicode), not an ASCII value. ASCII is only the first 128 characters of Unicode.

TIP

Use the wrapper constants instead of memorising ranges: `Integer.MAX_VALUE`, `Long.MIN_VALUE`, `Byte.MAX_VALUE`, `Double.MAX_VALUE`, `Character.MAX_VALUE`.

Literals

Literal	Example	Notes
Integer (decimal)	`int a = 255;`	An integer literal is `int` by default
Long	`long l = 9_000_000_000L;`	`L` suffix is required beyond the int range; prefer uppercase `L` (lowercase `l` looks like `1`)
Hexadecimal	`int h = 0xFF;`	255
Binary (Java 7+)	`int bin = 0b1111_1111;`	255
Octal	`int o = 0377;`	255. A leading `0` means octal, a common trap!
Underscores (Java 7+)	`long card = 1234_5678_9012_3456L;`	Readability only; not at the start or end, or next to `.`
Double	`double d = 3.14; 1.5e3`	A decimal literal is `double` by default
Float	`float f = 3.14f;`	`f`/`F` suffix is mandatory
Char	`'A'`, `'\n'`, `'\u0041'`	Single quotes, exactly one character
String	`"Hello"`	Double quotes; a reference type
Boolean	`true`, `false`	Not `1`/`0` as in C
Null	`null`	Default for every reference type

```
float a = 25.01;    // COMPILE ERROR: 25.01 is a double (8 bytes) -> possible lossy conversion
float b = 25.01f;   // OK
char rank = '1';    // OK: one character
char bad = '10';    // COMPILE ERROR: unclosed / too many characters in char literal
```

Escape sequences

Escape	Meaning	Escape	Meaning
`\n`	New line	`\\`	Backslash
`\t`	Tab	`\'`	Single quote
`\r`	Carriage return	`\"`	Double quote
`\b`	Backspace	`\uXXXX`	Unicode character (hex)
`\s`	Space (Java 15+)	`\0`	Null character (octal escape)

char is a number underneath

```
char c = 'A';
System.out.println(c + 1);          // 66   (char promoted to int)
System.out.println((char) (c + 1)); // B
System.out.println((int) 'a');      // 97
char u = '\u0041';                  // 'A' written as a Unicode escape
char next = c; next++;              // 'B' (++ on a char works)
```

Character	Code	Character	Code
'0' – '9'	48 – 57	'a' – 'z'	97 – 122
'A' – 'Z'	65 – 90	space	32

> **TIP**
> Convert a digit character to its number with `ch - '0'` (for example `'7' - '0'` is `7`), or `Character.getNumericValue(ch)`.

Floating-point precision and money

```
System.out.println(0.1 + 0.2);       // 0.30000000000000004
System.out.println(0.1 + 0.2 == 0.3);// false

import java.math.BigDecimal;
BigDecimal a = new BigDecimal("0.1");   // use the String constructor!
BigDecimal b = new BigDecimal("0.2");
System.out.println(a.add(b));           // 0.3
```

> **PITFALL**
> Never use `float` or `double` for currency. Binary floating point cannot represent most decimal fractions exactly. Use `BigDecimal` (built from a `String`, with an explicit `RoundingMode`), or store amounts as `long` minor units such as paise or cents. This is a standard banking and fintech interview question.

Integer overflow

```
int max = Integer.MAX_VALUE;          // 2147483647
System.out.println(max + 1);          // -2147483648  (silently wraps around!)
long ok = (long) max + 1;             // 2147483648   (widen BEFORE adding)
Math.addExact(max, 1);                // throws ArithmeticException: integer overflow
```

2.3 Type Conversion and Casting

Type casting converts a value from one type to another. There are two kinds for primitives.

Widening (implicit, automatic)

Smaller type → larger type. No cast is needed and the value always fits.

```
byte -> short -> int -> long -> float -> double
                  ^
                char
```

```
int i = 100;
long l = i;          // int  -> long   (automatic)
double d = l;        // long -> double (automatic)
int code = 'A';      // char -> int = 65
```

> **NOTE**
> `int → float` and `long → float/double` are "widening" but can **lose precision**, because a float has only about 7 significant digits. For example `(float) 123456789` stores 123456792 (prints `1.2345679E8`).

Narrowing (explicit, manual)

Larger type → smaller type. Requires a cast `(type)`, because data may be lost.

```
double d = 9.99;
int i = (int) d;           // 9    -> fractional part TRUNCATED, not rounded
long big = 130;
byte b = (byte) big;       // -126 -> overflow wraps around (130 - 256)
int r = (int) Math.round(9.99);   // 10 -> round when you mean round
char c = (char) 66;        // 'B'
```

Automatic type promotion in expressions

In arithmetic, `byte`, `short` and `char` are promoted to `int` first. The result type is the widest operand type (at least `int`).

```
byte a = 10, b = 20;
// byte c = a + b;      // COMPILE ERROR: a + b is an int
byte c = (byte) (a + b);// OK
int c2 = a + b;         // OK

byte x = 10;
x = x + 1;              // COMPILE ERROR (int result)
x += 1;                 // OK: compound assignment includes an implicit cast
x++;                    // OK

final byte k = 10;
byte y = k + 5;         // OK: compile-time constant that fits in a byte

long  total = 5 * 2L;   // long
double avg = 7 / 2;     // 3.0 -> integer division happens BEFORE widening!
double avg2 = 7 / 2.0;  // 3.5
```

Converting between String and primitives

Direction	Method	Example
String → int	`Integer.parseInt(s)`	`int n = Integer.parseInt("10");`
String → long / double / boolean	`Long.parseLong`, `Double.parseDouble`, `Boolean.parseBoolean`	`double d = Double.parseDouble("2.5");`
String → wrapper object	`Integer.valueOf(s)`	`Integer obj = Integer.valueOf("10");`
primitive → String	`String.valueOf(x)`	`String s = String.valueOf(10);`
primitive → String	`Integer.toString(x)` or `"" + x`	`String s = "" + 10;`
char → String	`String.valueOf(ch)` or `Character.toString(ch)`	`"A"`

```
Integer.parseInt("hello");       // NumberFormatException (unchecked)
Integer.parseInt(" 10");         // NumberFormatException: spaces are not trimmed
Integer.parseInt("ff", 16);      // 255: parse with a radix
Boolean.parseBoolean("yes");     // false: only "true" (any case) gives true
```

Command-line arguments

Arguments passed after the class name arrive in `main` as a `String[]`. They are always Strings.

```java
public class Calculator {
    public static void main(String[] args) {
        if (args.length < 2) {
            System.out.println("Usage: java Calculator <a> <b>");
            return;
        }
        System.out.println(args[0] + args[1]);          // "10" + "20" = 1020
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        System.out.println(a + b);                      // 30
    }
}
// Run:  java Calculator 10 20
// Arguments containing spaces need quotes:  java App "New Delhi"
```

PITFALL

Accessing `args[0]` with no arguments throws `ArrayIndexOutOfBoundsException`. Always check `args.length` first.

2.4 Reading Console Input

```java
import java.util.Scanner;

public class InputDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter age: ");
        int age = sc.nextInt();
        sc.nextLine();                    // consume the leftover newline!
        System.out.print("Enter name: ");
        String name = sc.nextLine();      // reads the whole line
        System.out.println(name + " is " + age);
        sc.close();
    }
}
```

Method	Reads
`nextInt()`, `nextLong()`, `nextDouble()`, `nextBoolean()`	One token converted to that type
`next()`	One word (stops at whitespace)
`nextLine()`	Rest of the current line
`hasNextInt()`, `hasNext()`	Checks before reading (validation)

PITFALL

After `nextInt()`, the newline stays in the buffer, so the next `nextLine()` returns an empty string. Add an extra `sc.nextLine()` as shown above.

2.5 Comments

Type	Syntax	Use
Single-line	`// comment`	Short notes; everything after `//` on that line is ignored
Multi-line	`/* ... */`	Longer explanations; cannot be nested
Documentation (Javadoc)	`/** ... */`	API docs placed before classes, methods, fields and constructors

```java
/**
 * Transfers money between two accounts.
 *
 * @param from   source account number
 * @param to     destination account number
 * @param amount amount to transfer; must be positive
 * @return the transaction reference id
 * @throws InsufficientFundsException if the balance is too low
 * @since 2.1
 */
public String transfer(String from, String to, BigDecimal amount) { ... }
```

Javadoc tag	Purpose
`@param`	Describes a parameter
`@return`	Describes the return value
`@throws` / `@exception`	Documents an exception
`@see` / `{@link Class#method}`	Cross-reference
`@since`, `@version`, `@author`	Version and author info
`@deprecated`	Marks as obsolete (pair with the `@Deprecated` annotation)
`{@code text}`	Shows inline code

Generate HTML documentation with `javadoc -d docs *.java`.

2.6 Identifiers and Keywords

An **identifier** is a name for a class, interface, method, variable or package.

Rules (compiler-enforced)

1. Allowed characters: letters `a-z`, `A-Z`, digits `0-9`, `_` and `$` (Unicode letters are also allowed).
2. Must **not start with a digit**.
3. Must not be a **reserved keyword** or the literals `true`, `false`, `null`.
4. **Case sensitive**: `name`, `Name` and `NAME` are three different identifiers.
5. **No spaces** or other symbols (`-`, `@`, `#`, ...).
6. No length limit. A lone `_` is not allowed since Java 9.

Identifier	Valid?	Reason
`age2`, `_bill`, `$amount`, `mobile_bill`	Valid	Allowed characters, doesn't start with a digit
`2age`	Invalid	Starts with a digit
`mobile bill`	Invalid	Contains a space
`total-amount`	Invalid	`-` is an operator
`class`, `int`	Invalid	Reserved keywords
`String`, `main`, `var`	Valid (but avoid)	Not keywords, just bad names

Reserved keywords

Category	Keywords
Primitive types & void	`byte short int long float double char boolean void`
Flow control	`if else switch case default while do for break continue return`
Modifiers	`public private protected static final abstract synchronized native transient volatile strictfp`
Exception handling	`try catch finally throw throws assert`
Class related	`class interface enum extends implements package import`
Object related	`new this super instanceof`
Reserved but unused	`const goto`
Underscore (Java 9+)	`_`

Literals (not keywords, but also reserved): `true`, `false`, `null`.

Contextual keywords have special meaning only in certain places and can still be used as identifiers elsewhere: `var`, `record`, `yield`, `sealed`, `permits`, `non-sealed`, `when`, and module words such as `module`, `requires` and `exports`.

> **CORRECTED FROM ORIGINAL NOTES**
> The original notes list `null` as a keyword and give the count as "53". Precisely: Java has 50 classic keywords plus _, and `true`, `false`, `null` are **literals**. Interviewers usually just want to know that `true`, `false` and `null` are literals rather than keywords, and that `const` and `goto` are reserved but unused.

2.7 Naming Conventions

Conventions are not enforced by the compiler but are followed everywhere. Methods and variables use *camelCase*; classes use *PascalCase*.

Element	Convention	Examples
Class / Interface / Enum / Record	PascalCase, nouns (interfaces often adjectives)	`PaymentService`, `Runnable`, `AccountType`
Method	camelCase, starts with a verb	`calculateInterest()`, `getBalance()`, `saveAllUsers()`
Variable	camelCase, meaningful noun	`userAge`, `creditCardNumber`
Boolean variable / method	Reads as a question	`isActive`, `hasBalance`, `canWithdraw()`
Constant (`static final`)	UPPER_SNAKE_CASE	`MIN_AGE`, `MAX_RETRY_COUNT`
Enum constant	UPPER_SNAKE_CASE	`SAVINGS`, `CURRENT`, `FIXED_DEPOSIT`
Package	all lowercase, reverse domain, dot separated	`java.lang`, `com.bank.payments.service`
Generic type parameter	Single uppercase letter	`T` (type), `E` (element), `K`, `V` (key, value), `R` (result)

> **TIP**
> Getters and setters follow the JavaBeans pattern: `getName()` / `setName()`, and `isActive()` for booleans. Frameworks such as Spring, Jackson and Hibernate rely on these names.

2.8 Operators

An **operator** is a symbol that performs an operation on **operands** (`a + b`: `+` is the operator, `a` and `b` are operands).

Type	Operators	Example
Arithmetic	`+ - * / %`	`a % b`
Unary	`+ - ++ -- ! ~`	`++a`, `-x`, `!flag`
Assignment	`= += -= *= /= %= &= |= ^= <<= >>= >>>=`	`a += 5`
Relational	`== != > < >= <=`	`a >= b`
Logical	`&& || !`	`a > 5 && b < 10`
Bitwise	`& | ^ ~`	`a & b`
Shift	`<< >> >>>`	`a << 2`
Ternary (conditional)	`condition ? x : y`	`(a > b) ? a : b`
Type comparison	`instanceof`	`obj instanceof String`
Member access	`.`	`System.out.println()`
Lambda / method reference	`-> ::`	`x -> x * 2`, `String::length`

CORRECTED FROM ORIGINAL NOTES

The original notes call `.` (dot) a relational operator. It is the **member access** operator, used to reach fields, methods and nested types.

Arithmetic surprises

```
System.out.println(7 / 2);      // 3      integer division truncates
System.out.println(7 % 3);      // 1
System.out.println(-7 % 3);     // -1     sign follows the LEFT operand
System.out.println(7.5 % 2);    // 1.5    % works on doubles too
System.out.println(10 / 0);     // ArithmeticException: / by zero
System.out.println(10.0 / 0);   // Infinity
System.out.println(0.0 / 0);    // NaN
System.out.println(Double.NaN == Double.NaN);   // false! use Double.isNaN(x)
```

Increment and decrement

Pre-increment (`++a`) increments first, then uses the value. **Post-increment** (`a++`) uses the current value, then increments.

```
int a = 5;
System.out.println(a++);  // 5   (prints, then a = 6)
a++;                      //     a = 7
System.out.println(a++);  // 7   (prints, then a = 8)
System.out.println(a);    // 8

int b = 8;
System.out.println(++b);  // 9
++b;                      //     b = 10
System.out.println(++b);  // 11
System.out.println(b);    // 11

int c = 3;
int d = ++c + c++;        // 4 + 4 = 8  (++c makes c 4; c++ uses 4 then c becomes 5)
System.out.println(d + " " + c);   // 8 5

int i = 5;
i = i++;                  // i stays 5! (old value is assigned back over the increment)
```

Relational and logical operators

```
int age = 25; boolean hasId = true;
boolean canEnter = age >= 18 && hasId;    // true
boolean discount = age < 12 || age > 60;  // false
boolean notAdult = !(age >= 18);          // false
```

Short-circuit evaluation: `&&` skips the right side if the left is `false`; `||` skips it if the left is `true`. The non-short-circuit `&` and `|` always evaluate both sides.

```
String s = null;
if (s != null && s.length() > 0) { }   // safe: length() is never called
if (s != null &  s.length() > 0) { }   // NullPointerException: both sides evaluated
```

> **TIP**
> Put the cheap or null-guarding condition first. Writing `"YES".equals(input)` instead of `input.equals("YES")` also avoids a `NullPointerException` when `input` is null.

Bitwise and shift operators

```
int a = 5;   // 0101
int b = 3;   // 0011
a & b   // 0001 = 1   AND: 1 only if both bits are 1
a | b   // 0111 = 7   OR:  1 if either bit is 1
a ^ b   // 0110 = 6   XOR: 1 if the bits differ
~a      // -6         NOT: flips every bit (~x == -x - 1)

8 << 1     // 16   shift left by n = multiply by 2^n
8 >> 1     // 4    shift right by n = divide by 2^n (keeps sign)
-8 >> 1    // -4   arithmetic shift: sign bit copied
-8 >>> 28  // 15   unsigned shift: fills with 0s

(n & 1) == 0          // even-number check
(x ^ y ^ y) == x      // XOR twice restores the value
(flags & MASK) != 0   // checks a permission/feature flag
```

Ternary operator

```
int max = (a > b) ? a : b;
String status = balance >= 0 ? "ACTIVE" : "OVERDRAWN";
String grade = marks >= 90 ? "A" : marks >= 75 ? "B" : "C";   // nested: keep it short
```

instanceof (with pattern matching, Java 16+)

```
Object obj = "rohit";
if (obj instanceof String) {               // classic: check, then cast
    String s = (String) obj;
    System.out.println(s.toUpperCase());
}
if (obj instanceof String s && s.length() > 3) {   // pattern matching: check + bind
    System.out.println(s.toUpperCase());
}
null instanceof String   // false: never throws
```

String concatenation with +

`+` is evaluated **left to right**. Once one operand is a `String`, it becomes concatenation.

```
System.out.println(1 + 2 + "3");     // "33"   (1+2 = 3, then "3"+"3")
System.out.println("1" + 2 + 3);     // "123"
System.out.println("1" + (2 + 3));   // "15"
System.out.println('a' + 'b');       // 195    (char + char = int!)
System.out.println("" + 'a' + 'b');  // "ab"
```

2.9 Operator Precedence

From highest to lowest. When in doubt, add parentheses: it costs nothing and helps readers.

Level	Operators	Associativity
1	Postfix `a++ a--`, member access `.`, `[]`, method call `()`	Left to right
2	Unary `++a --a + - ! ~`, cast `(type)`	Right to left
3	Multiplicative `* / %`	Left to right
4	Additive `+ -`	Left to right
5	Shift `<< >> >>>`	Left to right
6	Relational `< > <= >= instanceof`	Left to right
7	Equality `== !=`	Left to right
8	Bitwise AND `&`	Left to right
9	Bitwise XOR `^`	Left to right
10	Bitwise OR `|`	Left to right
11	Logical AND `&&`	Left to right
12	Logical OR `||`	Left to right
13	Ternary `? :`	Right to left
14	Assignment `= += -=` ... and lambda `->`	Right to left

```java
int r = 2 + 3 * 4;        // 14, not 20
boolean t = 5 > 3 == true; // (5 > 3) == true -> true
int x, y, z;
x = y = z = 10;           // assignment is right-associative
```

2.10 Predict the Output

```java
System.out.println(10 + 20 + "Java" + 10 + 20);   // 30Java1020
System.out.println('A' + 1);                      // 66
System.out.println((char) ('A' + 1));             // B
System.out.println(5 / 2 * 2.0);                  // 4.0  (5/2 = 2 first)
System.out.println(1 / 2 + 0.5);                  // 0.5
System.out.println((int) 3.99 + (int) -3.99);     // 0    (3 + -3, truncation toward 0)
System.out.println(Integer.MAX_VALUE + 1 < 0);    // true (overflow)
byte b = (byte) 200;  System.out.println(b);      // -56
int i = 0; i = i++ + ++i; System.out.println(i);  // 2    (0 + 2)
System.out.println(0.1f == 0.1);                  // false (float vs double precision)
long l = 1_000 * 1_000 * 1_000 * 10;              // overflow in int BEFORE widening!
System.out.println(l);                            // 1410065408 (use 10L)
```

2.11 Interview Questions

Q. What are primitive and reference data types?

Java has 8 primitives (`byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`) that hold the value directly. Reference types (classes, interfaces, arrays, `String`, enums, records) hold the address of an object on the heap; their default value is `null`.

Q. Why is `char` 2 bytes in Java but 1 byte in C?

Java uses Unicode (UTF-16) so it can represent characters from all languages, not just ASCII.

Q. What is the difference between widening and narrowing?
Widening converts a smaller type to a larger one automatically with no data loss (int → long). Narrowing converts a larger type to a smaller one, needs an explicit cast, and may truncate or overflow (`(int) 9.99` gives 9; `(byte) 130` gives -126).

Q. Why does `byte b = 10; b = b + 1;` fail but `b += 1;` compile?
`b + 1` is promoted to `int`, and assigning an `int` to a `byte` needs a cast. Compound assignment operators include an implicit cast back to the variable's type.

Q. Why shouldn't you use double for money?
Binary floating point can't exactly represent most decimal fractions (`0.1 + 0.2` gives `0.30000000000000004`). Use `BigDecimal` with explicit rounding, or `long` in the smallest currency unit.

Q. What is the difference between `&&` and `&`?
`&&` is logical AND with short-circuiting: the right side is skipped if the left is false. `&` always evaluates both sides and also works as bitwise AND on integers.

Q. What is the difference between `>>` and `>>>`?
`>>` is an arithmetic right shift that preserves the sign bit. `>>>` is an unsigned right shift that fills with zeros, so negative numbers become large positive ones.

Q. Are `true`, `false` and `null` keywords?
No, they are literals. They still cannot be used as identifiers.

Q. Do local variables get default values?
No. Instance and static variables get defaults; local variables must be explicitly initialised before use, otherwise it is a compile error.

Q. What is `var`? Does it make Java dynamically typed?
`var` (Java 10) lets the compiler infer a local variable's type from its initializer. The type is still fixed at compile time, so Java remains statically typed.

2.12 Quick Revision

- Declaration + initialization: `int a = 10;`. Locals have **no defaults**; fields do.
- Sizes in bytes: byte 1, short 2, int 4, long 8, float 4, double 8, char 2; boolean is JVM-dependent.
- Integer literals default to `int`; decimal literals default to `double`. Use the `L` and `f` suffixes.
- `0x` hex, `0b` binary, leading `0` octal; underscores allowed for readability.
- Widening: byte → short → int → long → float → double (char → int). Narrowing needs `(type)`.
- `byte`/`short`/`char` are promoted to `int` in expressions; compound assignment hides a cast.
- Integer `/` truncates; `/ 0` on ints throws; on doubles gives `Infinity`/`NaN`.
- Money: **BigDecimal**, never `double`.
- `a++` uses then increments; `++a` increments then uses.
- `&&`/`||` short-circuit; `&`/`|` don't.
- `+` evaluates left to right: `1 + 2 + "3"` is `"33"`.
- Naming: `PascalCase` classes, `camelCase` methods and variables, `UPPER_SNAKE` constants, lowercase packages.